

Capsa

One graph-shaped format for **projects**, **team**, and **company** — as readable files that live inside the repository itself, position and typed links carrying the structure.

- 1 What Capsa is, and what problem it solves
- 2 The record — the unit everything is built from
- 3 From tree to graph: containment and edges
- 4 What a project documents
- 5 Placement as scope: what an edge states, and what position already stated
- 6 Team and company: people, roles, and insights
- 7 How company documentation relates to project documentation
- 8 What's checked automatically — and what Capsa is not

An introduction for a technical reader who does not write code · no programming knowledge required

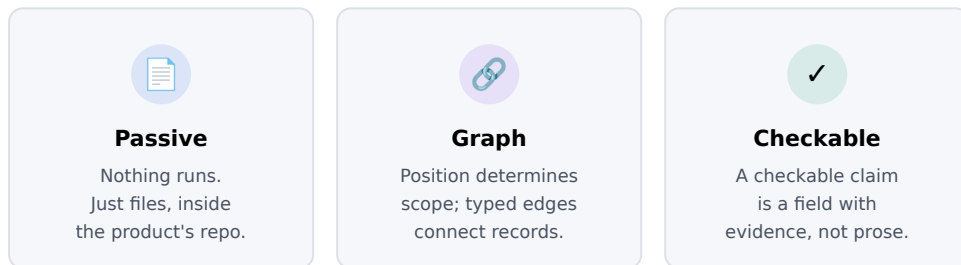
What Capsa is, and what problem it solves

Every project accumulates truth that isn't code: what's required, what was decided and why, what broke, what was learned. It's usually scattered — some in a task tool, some in chat, most in the head of whoever was there. When that person leaves, or a year passes, it's gone.

Capsa is a folder of readable text files that sits inside the product's own repository.

Every durable fact is a record — one file, readable by a person and parseable by a machine. There is no server, no database, no software that has to run for the documentation to exist: copy the project and the documentation comes with it; delete every tool and the documentation stays whole.

Three properties everything else follows from



The three properties everything else is derived from

A distinction worth holding through this document: Capsa documents **durable truth** — what's true about the project. It does **not** document run-state: who is working right now, what something cost, what's happening this minute. That's true for a moment and stale right after; the capsule is meant to be read three years from now.

The record — the unit everything is built from

Everything in Capsa is a record: one file, two parts. On top, ordered fields a machine reads; below, free text a person reads. One file serves both — so there are never two versions that can contradict each other.



One record — on top what's checked, below what's explained

The separation is the idea, not decoration

Notice the field `status: met` — "the requirement holds". In Capsa it cannot be written alone: it requires a `verification` block pointing at **the evidence** — the test that ran, the scan, the measurement. A claim with no evidence under it counts as "unverified" — a visible fact, not "probably fine".

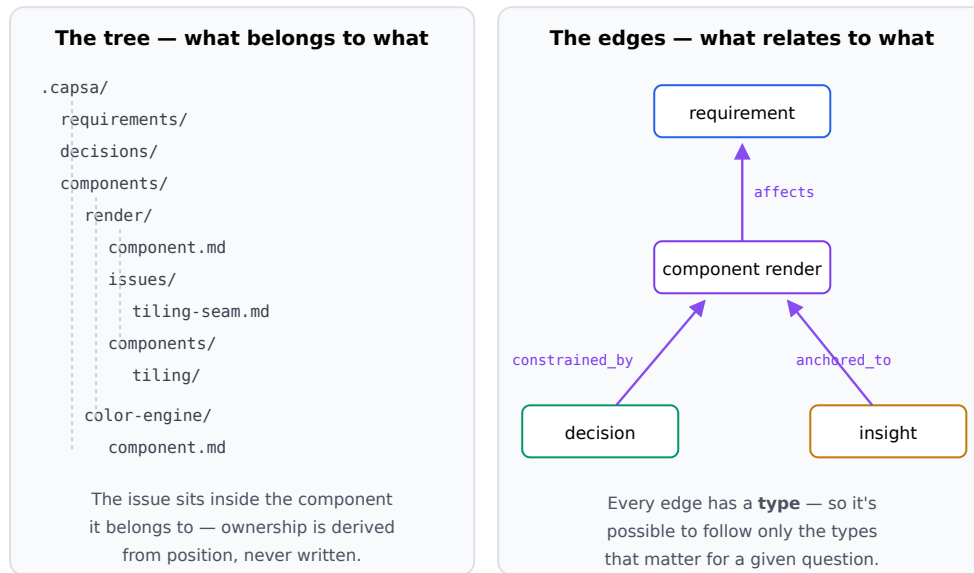
This rule is what separates documentation you can trust from documentation someone once wrote. It's also what lets a machine answer questions: *"Is every must-level requirement verified before release?"* is a query, not a survey.

Identity: the path is the name

A record is identified by **its path** — the directory and the file name. There's no central running number to allocate. That sounds technical, and it solves a very non-technical problem: when two people work in parallel, a running number guarantees a collision — both take "the next number". A path never collides.

From tree to graph: containment and edges

A capsule has two structures at once, exactly like code does: a **tree** that says what belongs to what, and **edges** that say what relates to what. The tree is the directories; the edges are explicit links inside records.



The same capsule, two structures: containment (left) and links (right)

Why files, not a graph database? Because every advantage of a graph is available here without giving up what files provide for free: full history, version comparison, change review, and backup — all already present in the version-control system the project uses anyway. The queryable graph is built from the files in memory when needed, and can be deleted and rebuilt. The files are the truth.

What a project documents

Record types aren't an arbitrary list — each is a kind of truth that behaves differently over time. There is no type for "platform", "language", or "regulation": a constraint on the code is a **requirement** of it, and where it has code of its own it's already a **component**. A format that grants its own type to one dimension would owe one to every dimension.

TYPE	WHAT IT IS	WHAT MAKES IT DISTINCT
Charter	The vision, constraints, ground rules	Written up front, rarely changes
Requirement	What the product must do	Needs evidence to count as "met"
Plan	An initiative that breaks into work	Finishes — unlike the project
Decision	An architectural call and its rationale	Never deleted or edited; replaced by a new decision
Discussion	Considerations that were weighed	Worth keeping even without a resolution
Issue	A bug, risk, or task	Closing it requires a fix <i>and a regression test</i>
Dependency	A third-party library and its license	Carries the facts that make compliance checkable
Release	What shipped, when, from which commit	Answers "what was in 2.3" years later
Insight	Learned knowledge — engineering, design, or code-anchored	Not a decision and not a task; knowledge
Component	Part of the system: a module, a subsystem	Answers "what's the architecture <i>right now</i> "
Interface	A contract others depend on	Has its own lifecycle: added · deprecated · removed
Milestone	A point in time several plans aim at	A date something can hang off
Release line	A maintenance stream (26.x live, 25.x fixes)	Lets a backport be recorded correctly

Two examples of a "fact that isn't scalar"

A requirement met only in part. "Met on Windows, not on iPad" isn't one value. Instead of rounding to one false answer, the record carries a general answer *and a list of exceptions*, and each exception **points at an existing record** — the requirement "run on iPad", or the component that implements it. The format has no closed list of dimensions: a product varies by version, by

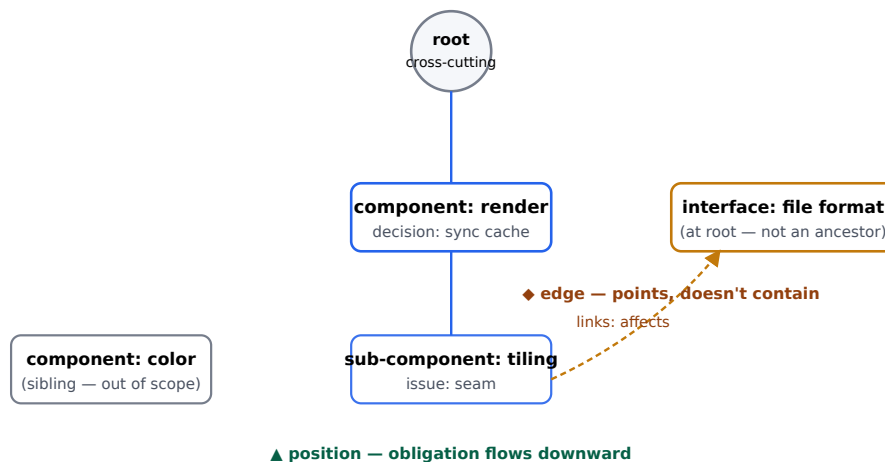
A fix that was backported. The same bug was fixed on the current line and on the maintained line — as two *different* commits. A single field can describe only one of them, so there's a list, per line.

subsystem, by regulation, by language, and by accessibility, and all of those are already recorded wherever they apply.

What all of these share: a checkable claim becomes a **field**, not a sentence. "No open severity-1 issue", "no dependency with an unapproved license", "every must-requirement is verified" — each becomes a question a machine answers in a second, instead of a manual review before every release.

Placement as scope: what an edge states, and what position already stated

In Capsa, a position in the tree carries meaning, not just an address: **a record applies to the node holding it and to everything beneath it**. A record at the root applies to the whole capsule. This is a fact about the graph, not a navigation convenience.



tiling inherits what applies to render and to the root; the edge to "interface" is not containment — it points

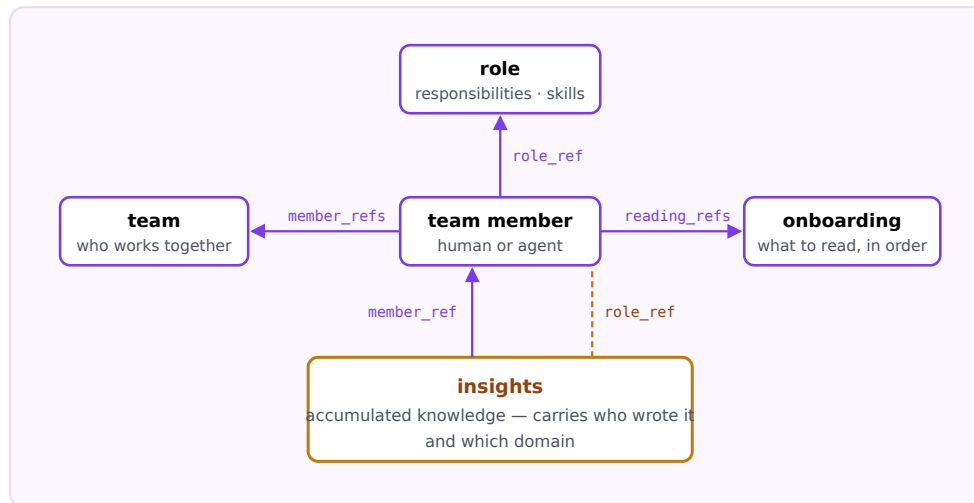
Two facts about the graph, not two-plus-one

- **Containment = scope.** "decision: sync cache" sits in component `render`, so it applies to it and to `tiling` inside it. No field states this — the path itself is the statement. Every record type is defined as either **binding** what's beneath it (requirement, decision, interface) or **only describing** (issue, insight, release).
- **An edge is what containment can't say.** "interface: file format" sits at the root, not an ancestor of `tiling` — so the relationship between `tiling` and it is an explicit edge, not inheritance. An edge that only restates what containment already said is invalid.

Addresses: absolute and relative. A link inside the same subtree is written relatively (`../capture/component`), so a subtree dragged with all its contents keeps working with no edits. A link crossing branches is written absolute from the root. A link crossing capsules is written `@capsule-slug/path` — and is allowed to stay unresolved, because a capsule remains whole on its own.

Team and company: people, roles, and insights

A company has its own truth, belonging to no single project: who works here, what each role is responsible for, what the organization has learned. This is the **organization format** — the same grammar, different record types, a separate home.



The organization format — an insight is connected both to who wrote it and to its domain

The property that matters: knowledge outlives turnover

When a team member leaves, their record isn't deleted — it moves to "departed" status. **The insights they wrote remain**, and stay connected to their domain. That's the difference between an organization where knowledge sits in people's heads and one where it's an asset.

Onboarding follows from this: an onboarding record is an ordered reading list built from knowledge already gathered. Whoever takes on a role gets what their predecessor learned, not a blank page. That's equally true for a new person and for an agent starting a shift — both are "team members" in the format, deliberately.

Two levels of knowledge

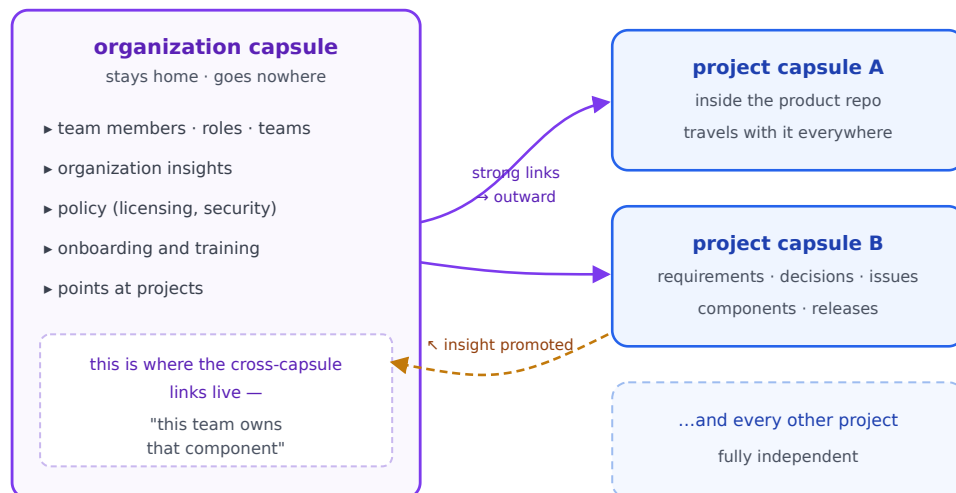
A role's insight. "When measuring render performance, the misleading metric is the average, not the 95th percentile." Belongs to the profession — relevant to whoever fills the role, on any project.

The company's insight. "Licensing decisions close with legal counsel before, not after, choosing the library." Crosses roles and projects — so it belongs to the organization as a whole.

The rule that prevents duplication: every fact has **one home**. An insight isn't copied between records — it's written once, and whoever needs it *points* at it. Two versions of the same fact are two versions that will contradict each other the day one of them is updated.

How company documentation relates to project documentation

Two capsules, two homes — deliberately. The separation isn't order for its own sake; it follows from three real boundaries.



The project travels; the organization stays. Strong links point outward from the side that stays home.

The three boundaries

- **Portability boundary.** A project capsule travels with the repo — clone, fork, handed to a client or contractor. If company knowledge sits inside it, it travels too.
- **Lifecycle boundary.** A project reaches archive; the company doesn't. Organizational knowledge must outlive the archiving of the project it was learned on.
- **Authority boundary.** Policy and roster shouldn't be editable by whoever happens to be working on one particular project.

And how they cross anyway

A cross-capsule address. An internal link **must** point at an existing record; an external one **may** stay unresolved — that's exactly the guarantee of portability: a project capsule stays valid and readable even when no other capsule is attached. It follows that strong links sit on the side that stays home.

What about an insight born in a project that turns out to be organizational? It **moves** to the organization capsule, and a "tombstone" stays behind — a short record saying where it went. Not a copy: one home per fact, and the history isn't cut off.

What's checked automatically — and what Capsa is not

The format ships with a read-only checker. It doesn't fix and doesn't write — it states what's non-conforming, in a language software can act on too.

CHECKED	WHAT IT PREVENTS IN PRACTICE
A requirement "met" with no evidence	A status someone updated hopefully
A closed bug with no fix and no regression test	A bug that returns and nobody knows it already happened
An internal link pointing at nothing	A graph that looks whole and isn't — the real danger
An exception pointing at a record that doesn't exist	A typo that silently means nothing
A link that restates what the path already says	A redundant edge someone has to maintain forever
An interface "removed" with no removal date	A consumer who discovers the removal the hard way
A dependency with a forbidden license and no admitting decision	Legal exposure nobody ever decided on

Every finding carries a **stable code** and a **severity**. The code is what software acts on; the human wording may change. A warning doesn't invalidate a capsule — it flags something worth doing, not something required.

What Capsa deliberately is not

- **Not a task-management tool.** No "who's working right now", no hours, no active board. That's run-state, true for a moment.
 - **Not metrics.** Anything derived from the records — counts, timelines, reports — is computed on demand, never stored.
- **Not code.** The capsule describes the project; the surrounding repository holds the product.
 - **Doesn't fix itself.** The checker only reads. Repair is a maintenance action — belonging to a tool someone runs, not to the format.

In one line: Capsa is the durable memory of the project and of the organization, kept as readable files nobody needs a tool to read — with position and typed links carrying enough structure that what belongs where is a fact, not a habit.